

Hierarchical Abstract Trees for Cross-Document Retrieval-Augmented Generation with CUDA-Accelerated GPU Computing

Dr. Kaushalya Thopate¹, Nivesh Jain², Hasan Rupawalla³, Harsh Nayak⁴, Rachit Ingole⁵

^{1,2,3,4,5}Department of Computer Engineering, Vishwakarma Institute of Technology, Pune-411037, Maharashtra, India.

kaushalya.thopate@vit.edu, nivesh.jain24@vit.edu, hasan.rupawalla24@vit.edu, harsh.nayak24@vit.edu, rachit.ingole24@vit.edu

Abstract: Retrieval-Augmented Generation (RAG) grounds large language models in external evidence, but the dominant flat dense-vector formulation scores every passage in the corpus independently and therefore fragments context: it answers narrow factual questions well while failing on queries whose evidence is distributed across several documents. Hierarchical methods such as RAPTOR recover part of the missing structure by recursively summarising passages into a tree, yet these trees are normally constructed per document and consequently encode no explicit relation between documents. This paper presents HAT-RAG, a cross-document Hierarchical Abstract Tree in which recursive GPU-accelerated clustering and abstractive summarisation are applied to the pooled chunk population of the entire corpus, so that every abstract node summarises material drawn from several source documents and explicit cross-document edges connect semantically related nodes at every level. Retrieval is a beam-guided top-down traversal of this tree, followed by a diversity-aware re-ranking step that rewards evidence originating from documents not yet represented in the retrieved set. All similarity computation, clustering and embedding is dispatched to batched tensor kernels that execute on NVIDIA CUDA devices when available and degrade to equivalent CPU kernels otherwise. The system is evaluated on a synthetic cross-document benchmark of up to 1,647 chunks with planted single-hop and two-hop evidence, against flat dense retrieval, a RAPTOR-style collapsed-tree baseline and an entity-graph baseline. Across a sixteen-fold increase in corpus size, HAT-RAG scores 60.5% to 95.3% fewer candidate nodes per query than flat dense retrieval — tracking the derived $\mathcal{O}(\beta b \log_b N)$ bound to within 2.5% — and its retrieval latency becomes effectively independent of corpus size, ending $7.90\times$ faster than flat retrieval at the largest scale. The diversity term raises cross-document evidence coverage from 54.2% to 95.8% with no loss of single-hop accuracy, HAT-RAG attains the highest cross-document coverage of the four architectures compared (93.8%), and the batched similarity kernel is measured at $124\times$ the throughput of a scalar reference implementation. A beam-width study maps the resulting efficiency–accuracy frontier, recovering an 86.1% evidence hit rate at 17.6% of the scoring cost of exhaustive search, and we state explicitly where hierarchical traversal does not pay off: exhaustive scoring retains the accuracy ceiling for narrow single-hop queries.

Keywords: Retrieval-Augmented Generation; Hierarchical Retrieval; Cross-Document Question Answering; Abstractive Summarisation; GPU Acceleration; NVIDIA CUDA; Dense Vector Search; Large Language Models.

1 Introduction

Large language models (LLMs) have become the default interface to unstructured text, but their parametric knowledge is frozen at training time, is expensive to refresh, and cannot cover private or newly created documents. Retrieval-Augmented Generation (RAG) [1, 2] addresses this by retrieving evidence from an external corpus at inference time and conditioning generation on the retrieved material, which both grounds the answer and makes it auditable through citations. The formulation has been refined along many axes — dense encoders [3, 4], hypothetical document embeddings [5], fusion-in-decoder readers [6], self-reflective and corrective control loops [8, 9], and adaptive routing between retrieval strategies [10] — and is surveyed comprehensively in [11, 12].

Despite this progress, the retrieval substrate underneath most deployed systems is unchanged: the corpus is split into chunks, each chunk is embedded independently, and a query is answered by scoring every chunk embedding and returning the k nearest. This *flat* formulation has two structural consequences. First, cost grows linearly in corpus size, because the number of similarity evaluations equals the number of chunks; approximate indices [30, 31] reduce the constant factor but preserve the flat view of the corpus. Second, and more fundamentally, a flat index has no representation of context above the chunk. A query such as “*how does the thermal-management strategy of programme A relate to the qualification regime described for programme B*” has no single chunk that answers it; the evidence is distributed over two documents, and a ranker that scores chunks independently has no mechanism for preferring a complementary pair of chunks over two near-duplicate chunks from the same document. The well-documented tendency of long-context readers to under-use material placed in the middle of their context window [27] makes indiscriminately enlarging k an unsatisfying remedy.

Hierarchical retrieval addresses the second problem directly. RAPTOR [13] recursively clusters chunks, summarises each cluster with an LLM, and repeats until a small number of root abstracts remain, so a query can be matched against nodes at whichever level of granularity suits it. Graph-based approaches instead extract entities and relations and reason over the resulting knowledge graph [14, 15, 16], which is effective for multi-hop entity questions but depends on the quality of extraction and becomes expensive to build and traverse as the corpus grows. The gap this paper addresses lies between the two: tree methods are typically constructed *within* a document and therefore encode no explicit relation across documents, while graph methods capture cross-document relations only implicitly, through shared entity mentions, and are poorly suited to producing fluent corpus-level abstractions.

We propose the **Hierarchical Abstract Tree (HAT)**, a single index built over the pooled chunk population of an entire corpus. Clustering at every level is performed across document boundaries, so an abstract node summarises material that may originate from a dozen different documents, and explicit cross-document edges link nodes whose embeddings exceed a similarity threshold. Queries are answered by a beam-guided top-down traversal that scores only the children of surviving branches, and the resulting candidate set is re-ranked with a diversity-aware objective that explicitly rewards evidence from documents not yet represented. Because tree construction, embedding and traversal are dominated by dense linear algebra, every such stage is expressed as a batched tensor operation dispatched to an NVIDIA CUDA device when one is present, with an equivalent CPU kernel and a dependency-free scalar fallback otherwise.

The contributions of this work are:

1. A **cross-document hierarchical index**, in which recursive semantic clustering and abstractive summarisation are applied to the pooled chunk population of the whole corpus rather than per document, giving a single structure that serves token-level, document-level and corpus-level queries (Section 4).
2. A **beam-guided top-down retrieval algorithm** with an explicit complexity analysis showing that the number of scored nodes falls from $\mathcal{O}(N)$ to $\mathcal{O}(\beta b \log_b N)$ for branching factor b and beam width β , validated empirically up to $N = 1,647$ (Sections 4 and 6).
3. A **diversity-aware cross-document ranking** term that raises measured cross-document evidence cov-

erage from 54.2% to 95.8% with no measurable loss of single-hop accuracy, with an empirically identified operating point at $\lambda \approx 0.2$ and a degradation of only 2.5 percentage points once $\lambda \geq 0.5$.

4. A **device-agnostic CUDA acceleration layer** whose batched similarity kernel is measured at $124\times$ the throughput of a scalar reference implementation, together with an open implementation comprising an ingestion pipeline, a Streamlit analyst interface and a FastAPI microservice.
5. A **controlled empirical study** against flat dense retrieval, a RAPTOR-style collapsed-tree baseline and an entity-graph baseline on a benchmark with planted single-hop and two-hop evidence, including an explicit account of the configurations in which hierarchical traversal loses accuracy or fails to reduce wall-clock latency.

The remainder of the paper is organised as follows. Section 2 reviews related work and positions the proposed method. Section 3 describes the system architecture and its data flow. Section 4 develops the mathematical formulation and the four algorithms of the pipeline. Section 5 details the experimental setup, benchmark and metrics. Section 6 reports and discusses the results. Section 7 concludes and outlines future work.

2 Related Work

2.1 Flat dense retrieval-augmented generation

The original RAG formulation [1] couples a dense passage retriever with a sequence-to-sequence reader, jointly grounding generation in retrieved passages; REALM [2] and RETRO [7] explore related pre-training-time formulations. Dense Passage Retrieval [3] established the dual-encoder recipe that most systems still use, ColBERT [4] refined it with late interaction, and HyDE [5] improved zero-shot recall by embedding a hypothetical answer rather than the raw query. Sentence-level encoders [36] and the transformer architectures beneath them [34, 35] supply the embedding functions used in practice, while lexical scoring [37] remains a strong hybrid component. Work on retrieval granularity [22] shows that the choice of indexing unit — sentence, proposition or passage — materially changes downstream accuracy, which is precisely the choice a hierarchical index defers to query time instead of fixing at index time.

2.2 Hierarchical and tree-structured retrieval

RAPTOR [13] is the closest antecedent of this work: it recursively clusters chunk embeddings, summarises each cluster with an LLM and repeats, yielding a tree whose upper levels are abstractive summaries. Queries may be matched against a collapsed pool of all nodes or by tree traversal. StructRAG [19] and hierarchical context partitioning [20] pursue related structuring strategies, and tree-structured prompting has been combined with retrieval in [18]. HiRAG [17] applies hierarchical organisation to the retrieval index as well. What these methods share is that the hierarchy is built over a single document or that cross-document relationships, if present at all, arise only incidentally from clustering; none makes cross-document linkage an explicit, first-class element of the index, and none is designed around GPU-resident execution.

2.3 Graph-structured retrieval

GraphRAG [14] builds an entity-relation graph over the corpus and answers global, query-focused summarisation questions by summarising graph communities. HippoRAG [15] draws on models of hippocampal indexing and applies personalised PageRank over a knowledge graph to support multi-hop retrieval, and LightRAG [16] reduces the cost of graph construction and query. Surveys of knowledge-graph-augmented language models [21] document the breadth of this family. Graph methods capture cross-document structure through shared entities, but their quality is bounded by entity and relation extraction,

graph construction cost grows super-linearly in the worst case, and pure graph structures are less suited to producing fluent document-level abstractions — a limitation our own entity-graph baseline reproduces in Section 6.

2.4 Adaptive, corrective and long-context strategies

A parallel line of work improves *when* and *how* retrieval is invoked rather than how the index is organised. Self-RAG [8] trains a model to decide when to retrieve and to critique its own output; corrective RAG [9] adds a retrieval evaluator that triggers correction when evidence is judged insufficient; Adaptive-RAG [10] routes queries between strategies by predicted complexity; active retrieval [23] retrieves as generation proceeds. Long-context approaches [25, 26, 24] enlarge the evidence window instead, but the observed degradation of attention to mid-context material [27] limits how far this scales. These strategies are orthogonal to, and composable with, the index proposed here.

2.5 GPU-accelerated vector search and systems support

Billion-scale similarity search on GPUs [30] and graph-based approximate nearest-neighbour indices [31] established that the throughput of the similarity kernel, not the ranking model, governs large-scale retrieval latency; more recent work targets high-dimensional GPU indexing [28] and fast GPU similarity search for multi-document retrieval [29] directly. On the generation side, quantised fine-tuning [32] and accelerated decoding [33] reduce the cost of the reader. Our system inherits the same premise — that the dense linear algebra should be batched and device-resident — and applies it to every stage of a hierarchical index, including the clustering used to build it [39, 40].

2.6 Evaluation of RAG systems

Reference-free and automated evaluation frameworks including RAGAS [42], ARES [43] and RAGBench [44], together with cross-document summarisation benchmarks [45] and classical summarisation metrics [41], define current practice. Because published cross-document benchmarks do not expose the identity of the gold evidence *chunk* — which is what an index comparison must measure — we complement them with a synthetic benchmark in which the gold chunks and the required cross-document hops are known by construction (Section 5).

2.7 Research gap

Table 1 summarises the comparison. No existing family combines (i) explicit cross-document abstractive structure, (ii) a single index serving token-level, document-level and corpus-level queries, (iii) an explicit diversity mechanism for assembling complementary evidence, and (iv) an architecture designed from the outset for GPU-resident execution. HAT-RAG is designed to occupy that intersection.

Table 1: Positioning of HAT-RAG with respect to the principal RAG index families. “Explicit” means the property is a first-class element of the index rather than an emergent side effect.

Aspect	Flat dense RAG	RAPTOR / tree	GraphRAG HippoRAG	/	HAT-RAG (proposed)
Index scope	Chunk pool	Per-document tree	Corpus graph	entity	Cross-document abstract tree
Granularity	Chunk only	Chunk to document abstract	Entity–relation triples		Chunk to corpus-level abstract
Cross-document links	None	Not explicit	Implicit via shared entities		Explicit typed edges
Query types	Token-level	Mostly document-level	Multi-hop entity QA	entity	Token-, document- and corpus-level
Evidence diversity	Not modelled	Not modelled	Graph-walk dependent	de- pendent	Explicit λ -weighted term
Nodes scored / query	$\mathcal{O}(N)$	$\mathcal{O}(N_{\text{all levels}})$	$\mathcal{O}(V+E)$		$\mathcal{O}(\beta b \log_b N)$
GPU acceleration	Index-level only	Not emphasised	Not emphasised		Batched CUDA kernels throughout

3 System Architecture

Figure 1 shows the architecture of HAT-RAG. The system separates cleanly into an *offline knowledge-construction* path, executed once per corpus revision, and an *online query-and-answer* path, executed per request. The two paths meet at the Hierarchical Abstract Tree H , which is the only persistent artefact shared between them. Figure 2 expands the same pipeline into the operational flow of the four algorithms that Section 4 formalises.

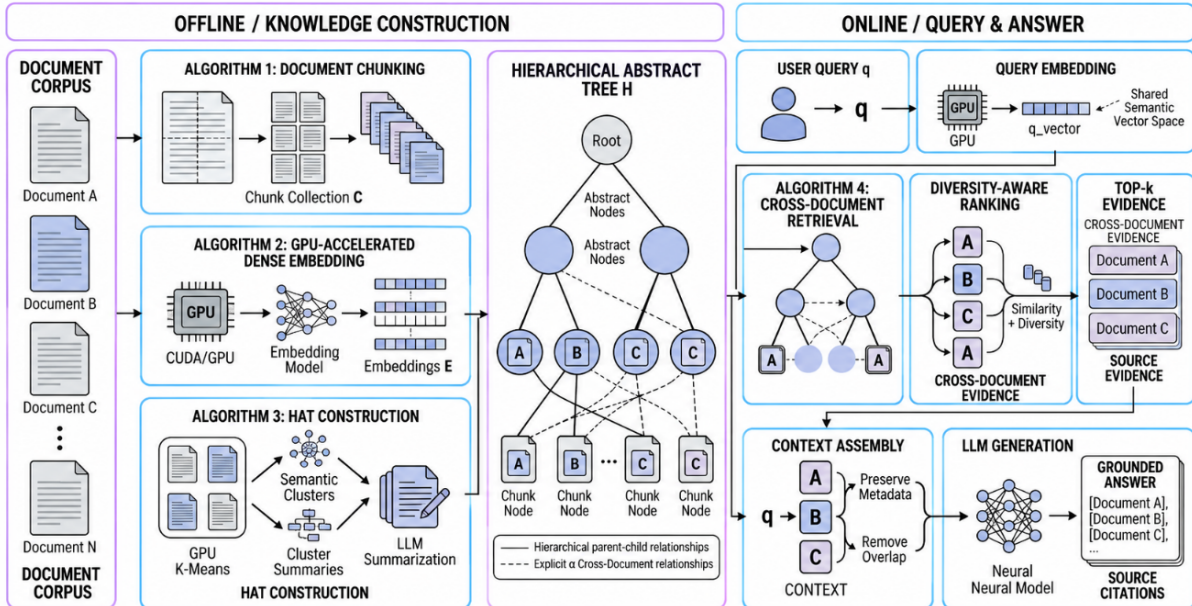


Figure 1: System architecture of HAT-RAG. The offline path (left) chunks the corpus, embeds chunks in batches on the GPU, and recursively clusters and summarises them into the Hierarchical Abstract Tree H , whose solid edges are parent–child relations and whose dashed edges are explicit cross-document relations. The online path (right) embeds the query in the same vector space, traverses H top-down, applies diversity-aware ranking to assemble cross-document evidence, and conditions the generator on the assembled context so that every claim carries a source citation.

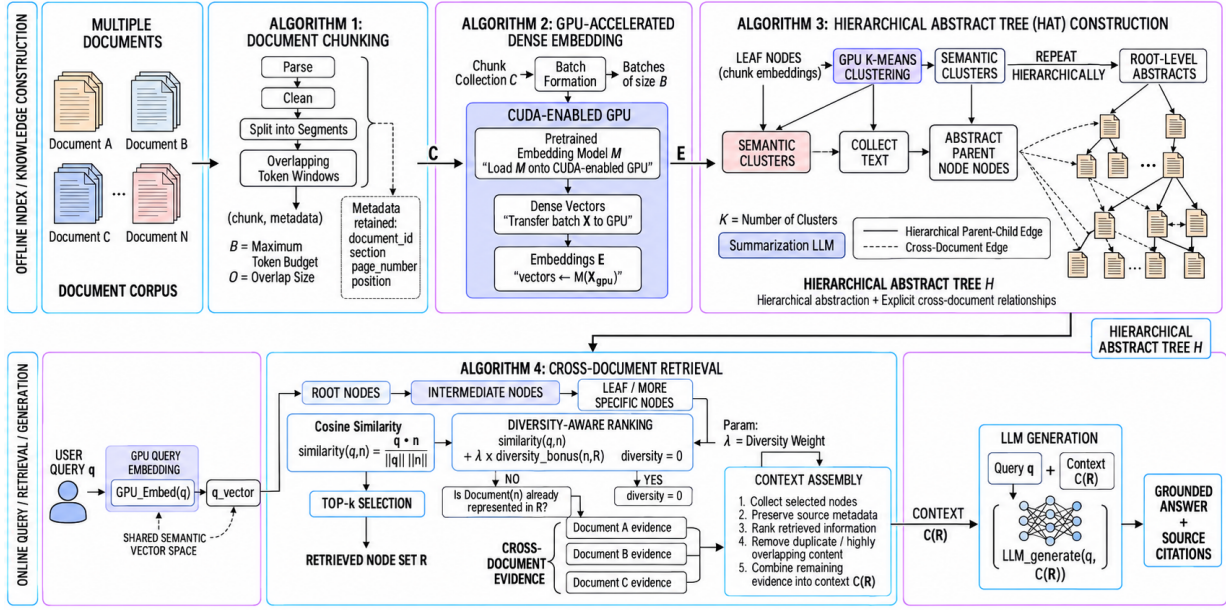


Figure 2: End-to-end process flow. Algorithm 1 produces the chunk collection $\mathcal{C}$ with retained provenance metadata; Algorithm 2 maps $\mathcal{C}$ to the embedding matrix E using batched CUDA inference; Algorithm 3 builds the tree H by repeated GPU k -means clustering and LLM summarisation; Algorithm 4 performs the online traversal, diversity-aware ranking, context assembly and grounded generation.

3.1 Ingestion layer

The ingestion layer parses heterogeneous sources (PDF, plain text, HTML, logs), normalises the extracted text and splits it into overlapping token windows. Each resulting chunk retains the provenance metadata — document identifier, section, page and position — that the generation layer later converts into citations. Overlap is essential: without it, an assertion straddling a window boundary is recoverable from neither window. In the reference implementation this layer is `document_processor.py`.

3.2 Indexing layer: HAT construction

The indexing layer embeds every chunk with a sentence encoder executing on the GPU, clusters the resulting vectors *across document boundaries*, and summarises each cluster into a parent node whose text is an abstractive summary of its children and whose embedding is computed in the same space. The procedure recurses on the parent population until the surviving node count falls below the branching factor. Two edge types are produced: parent-child edges, which constitute the tree, and cross-document edges, which connect nodes whose mutual similarity exceeds a threshold and whose provenance differs. This layer corresponds to `embeddings.py`, `summarizer.py` and `hierarchical_tree.py`; the resulting structure serialises to JSON so that an index survives process restarts.

3.3 Retrieval layer

The retrieval layer embeds the incoming query with the same encoder — which is what makes query and node embeddings comparable — and descends the tree from the root abstracts, expanding only the children of the β highest-scoring nodes at each level. Broad summarisation queries can be answered by halting at an upper level, and narrow factual queries by descending to the leaves; the level at which traversal stops is therefore a property of the query, not of the index. Candidate leaves are re-ranked with the diversity-aware objective of Section 4.6. This layer is `retriever.py`, with the comparative baselines in `multi_approach.py`.

3.4 Generation layer

The generation layer de-duplicates overlapping evidence, assembles the surviving nodes into a single prompt context ordered by rank, and invokes the generator. Because every node carries provenance, each fragment of the assembled context is attributable, and the answer is returned together with the citation list that supports it (`generator.py`).

3.5 Acceleration layer

Cutting across all four layers, the acceleration layer (`cuda_utils.py`) provides device detection, memory accounting and the batched cosine-similarity kernel. It dispatches to one of three backends, in order of preference: PyTorch tensors resident on an NVIDIA CUDA device; vectorised host tensors; or a dependency-free scalar implementation. The three backends are numerically equivalent, so an index built on one machine is portable to another, and only throughput changes. Section 6.7 quantifies that difference.

3.6 Service and interface layer

The system is exposed through a FastAPI microservice (`api.py`) with `/health`, `/ingest`, `/query` and `/benchmark` endpoints, and through a Streamlit analyst dashboard (`app.py`) that visualises the tree, the traversal path taken for a given query, and the live comparison against the baseline architectures.

4 Methodology

This section formalises the four stages of the pipeline. Throughout, the corpus is $\mathcal{D} = \{d_1, \dots, d_M\}$ with M documents, $f_\theta : \Sigma^* \rightarrow \mathbb{R}^D$ is the sentence encoder with embedding dimension D , and $\mathcal{S}$ denotes an abstractive summariser. Table 2 in Section 5 lists the symbols together with the values used in the experiments.

4.1 Problem formulation

Given a query q and a corpus $\mathcal{D}$, the objective is to select an evidence set R and to generate an answer y conditioned on it, such that (i) R contains the passages required to support y , (ii) R spans every document that contributes to the answer when the query is inherently cross-document, and (iii) the number of candidate representations scored in order to obtain R is sub-linear in the corpus size. Flat dense retrieval satisfies (i) but violates (ii) and (iii); per-document trees satisfy (iii) but violate (ii). The index developed below is designed to satisfy all three simultaneously.

4.2 Document ingestion and chunking

Each document d_m is tokenised into a sequence $t_{1:T_m}^{(m)}$ and split into overlapping windows of budget B tokens with stride $B - O$, where O is the overlap. The chunk collection is

$$\mathcal{C} = \left\{ c_{m,i} = t_{s_i:s_i+B}^{(m)} \mid s_i = i(B - O), 0 \leq i < \left\lceil \frac{T_m - O}{B - O} \right\rceil, 1 \leq m \leq M \right\}, \quad (1)$$

with $N = |\mathcal{C}|$ the total number of leaf chunks. Every chunk retains the provenance tuple $\pi(c) = \langle \text{doc_id}, \text{section}, \text{page}, \text{position} \rangle$ which is the mechanism by which the final answer is made citable. The overlap $O > 0$ guarantees that any assertion shorter than O tokens is fully contained in at least one window.

4.3 GPU-accelerated dense embedding

Chunks are embedded in batches of size B_{gpu} so that the encoder executes as a sequence of large matrix multiplications rather than N independent forward passes. For a batch $\mathbf{X}$ of B_{gpu} chunk strings transferred to device memory, the embedding matrix is

$$\mathbf{E} = [\hat{f}_\theta(x_1); \hat{f}_\theta(x_2); \dots; \hat{f}_\theta(x_{B_{\text{gpu}}})]^\top \in \mathbb{R}^{B_{\text{gpu}} \times D}, \quad \hat{f}_\theta(x) = \frac{f_\theta(x)}{\|f_\theta(x)\|_2 + \varepsilon}, \quad (2)$$

where $\varepsilon = 10^{-9}$ guards against division by zero for degenerate inputs. Because the rows of $\mathbf{E}$ are ℓ_2 -normalised, the similarity between a query embedding $\mathbf{q}$ and all n candidate embeddings $\hat{\mathbf{E}}$ reduces to a single matrix–vector product,

$$\text{sim}(q, \mathcal{N}) = \hat{\mathbf{q}} \hat{\mathbf{E}}^\top \in \mathbb{R}^n, \quad \text{sim}(q, n_j) = \frac{\mathbf{q} \cdot \mathbf{n}_j}{\|\mathbf{q}\|_2 \|\mathbf{n}_j\|_2}, \quad (3)$$

which is exactly the kernel dispatched to the CUDA device by `cuda_utils.gpu_batch_cosine_similarity`. Expressing scoring in the form of Eq. (3) rather than as a loop over candidates is what allows an entire tree level to be scored in one kernel launch, and it is the single design decision responsible for the throughput reported in Section 6.7.

4.4 Hierarchical Abstract Tree construction

Let $\mathcal{V}_0 = \mathcal{C}$ be the leaf population. At level ℓ , the node population $\mathcal{V}_\ell$ is partitioned into $K_\ell = \lceil |\mathcal{V}_\ell|/b \rceil$ clusters by minimising the within-cluster sum of squares over the embeddings, with b the branching factor:

$$\{\mathcal{P}_1^{(\ell)}, \dots, \mathcal{P}_{K_\ell}^{(\ell)}\} = \arg \min_{\{\mathcal{P}_k\}} \sum_{k=1}^{K_\ell} \sum_{v \in \mathcal{P}_k} \|\mathbf{z}_v - \mu_k\|_2^2, \quad \mu_k = \frac{1}{|\mathcal{P}_k|} \sum_{v \in \mathcal{P}_k} \mathbf{z}_v, \quad (4)$$

solved by Lloyd’s algorithm with k -means++ seeding [39, 40]. Crucially, the partition in Eq. (4) is computed over the *pooled* population of all documents; membership of a cluster is determined by semantics alone, so a single cluster routinely contains chunks drawn from many different source documents. This is the point at which HAT departs from per-document tree construction.

Each cluster induces one parent node whose surface form is an abstractive summary of its children and whose embedding is obtained by encoding that summary in the same space:

$$\text{text}(u_k^{(\ell+1)}) = \mathcal{S}\left(\{\text{text}(v)\}_{v \in \mathcal{P}_k^{(\ell)}}\right), \quad \mathbf{z}_{u_k} = \hat{f}_\theta(\text{text}(u_k)), \quad \text{ch}(u_k) = \mathcal{P}_k^{(\ell)}. \quad (5)$$

Encoding the *summary* rather than averaging child embeddings is deliberate: the centroid of a heterogeneous cluster is not in general a point that the encoder would assign to any real text, whereas the embedding of a written abstract is, which keeps upper-level nodes directly comparable to queries. The recursion terminates when $|\mathcal{V}_{\ell+1}| \leq b$ or a level budget L_{max} is reached, giving the tree

$$H = (\mathcal{V}, \mathcal{E}_{\text{tree}} \cup \mathcal{E}_{\text{doc}}), \quad \mathcal{V} = \bigcup_{\ell=0}^L \mathcal{V}_\ell, \quad \mathcal{E}_{\text{doc}} = \{(u, v) : \text{sim}(u, v) \geq \tau, \text{doc}(u) \neq \text{doc}(v)\}, \quad (6)$$

where $\mathcal{E}_{\text{tree}}$ holds the parent–child relations of Eq. (5) and $\mathcal{E}_{\text{doc}}$ holds the explicit cross-document edges, admitted above a similarity threshold τ . Since $|\mathcal{V}_{\ell+1}| \approx |\mathcal{V}_\ell|/b$, the depth satisfies $L = \lceil \log_b N \rceil$ and the total node count is bounded by $|\mathcal{V}| < Nb/(b-1)$: the index adds a bounded constant-factor overhead over the leaf population, 14.3% for $b = 8$, which agrees with the measured values in Table 3. Algorithm 1 states the procedure.

Algorithm 1 Cross-document Hierarchical Abstract Tree construction

Require: chunk collection $\mathcal{C}$, branching factor b , level budget $L_{\max}$, threshold τ

Ensure: tree $H = (\mathcal{V}, \mathcal{E}_{\text{tree}} \cup \mathcal{E}_{\text{xdoc}})$

```

1:  $\mathbf{E}_0 \leftarrow \text{BATCHEMBEDGPU}(\mathcal{C})$  ▷ Eq. (2), batched CUDA inference
2:  $\mathcal{V}_0 \leftarrow \{ \langle c, \mathbf{z}_c, \pi(c), \ell=0 \rangle : c \in \mathcal{C} \}; \quad \ell \leftarrow 0$ 
3: while  $\ell < L_{\max}$  and  $|\mathcal{V}_\ell| > b$  do
4:    $K \leftarrow \lceil |\mathcal{V}_\ell| / b \rceil$ 
5:    $\{ \mathcal{P}_k \}_{k=1}^K \leftarrow \text{KMEANSGPU}(\{ \mathbf{z}_v \}_{v \in \mathcal{V}_\ell}, K)$  ▷ Eq. (4), pooled over all documents
6:    $\mathcal{V}_{\ell+1} \leftarrow \emptyset$ 
7:   for all  $k \in \{1, \dots, K\}$  do
8:      $s_k \leftarrow \mathcal{S}(\{ \text{text}(v) \}_{v \in \mathcal{P}_k})$  ▷ abstract of a cross-document cluster
9:      $u_k \leftarrow \langle s_k, \hat{f}_\theta(s_k), \ell+1 \rangle; \quad \text{ch}(u_k) \leftarrow \mathcal{P}_k$ 
10:     $\mathcal{V}_{\ell+1} \leftarrow \mathcal{V}_{\ell+1} \cup \{u_k\}$ 
11:   end for
12:    $\ell \leftarrow \ell + 1$ 
13: end while
14:  $\mathcal{E}_{\text{xdoc}} \leftarrow \{ (u, v) \in \mathcal{V}^2 : \text{sim}(u, v) \geq \tau, \text{doc}(u) \neq \text{doc}(v) \}$ 
15: return  $H$  with root population  $\mathcal{V}_\ell$ 
    
```

4.5 Beam-guided top-down retrieval

Retrieval starts at the root population $\mathcal{V}_L$ and descends one level at a time. Writing $\mathcal{F}_\ell$ for the frontier retained at level ℓ , the candidate population at the next level is the union of the children of the frontier, and the next frontier is its β highest-scoring members:

$$\mathcal{A}_{\ell-1} = \bigcup_{u \in \mathcal{F}_\ell} \text{ch}(u), \quad \mathcal{F}_{\ell-1} = \arg \max_{\mathcal{F} \subseteq \mathcal{A}_{\ell-1}, |\mathcal{F}|=\beta} \sum_{v \in \mathcal{F}} \text{sim}(q, v), \quad \mathcal{F}_L = \mathcal{V}_L, \quad (7)$$

with β the beam width. Scoring of $\mathcal{A}_{\ell-1}$ is a single invocation of Eq. (3), so one kernel launch is issued per level rather than per node. The descent halts either at the leaves, giving passage-level evidence, or at the abstraction level requested by the query, giving summary-level evidence from the same index — the property that allows a single structure to serve both narrow and broad questions.

4.6 Diversity-aware cross-document ranking

Pure similarity ranking is biased towards near-duplicate evidence: when one document discusses a topic at length, its chunks occupy the entire top- k list and the complementary passage in another document is displaced. Following the maximal-marginal-relevance principle [38], the retrieved set is therefore built greedily under a composite objective,

$$\text{score}(q, v \mid R) = \text{sim}(q, v) + \lambda \text{div}(v, R), \quad R \leftarrow R \cup \left\{ \arg \max_{v \in \mathcal{A} \setminus R} \text{score}(q, v \mid R) \right\}, \quad (8)$$

where the diversity bonus rewards provenance not yet represented in the partially constructed set,

$$\text{div}(v, R) = \begin{cases} 1, & \text{if } \text{doc}(v) \notin \{ \text{doc}(r) : r \in R \}, \\ 0, & \text{otherwise,} \end{cases} \quad (9)$$

and $\lambda \geq 0$ trades evidence breadth against per-passage relevance. Setting $\lambda = 0$ recovers pure similarity ranking; Section 6.6 shows that cross-document coverage rises steeply for $\lambda \in (0, 0.2]$ and saturates thereafter, while single-hop accuracy is unaffected until $\lambda > 0.3$.

4.7 Context assembly and grounded generation

The selected nodes are de-duplicated, ordered by score and concatenated with their provenance markers into a context $C(R)$ subject to the reader token budget $T_{\max}$, and the answer is produced by conditioning the generator on that context:

$$C(R) = \bigoplus_{v \in R} \langle \pi(v), \text{text}(v) \rangle \quad \text{s.t.} \quad |C(R)| \leq T_{\max}, \quad y = \text{LLM}(q, C(R)), \quad (10)$$

where $\bigoplus$ denotes ordered concatenation. Because $\pi(v)$ travels with the text, every span of the generated answer can be traced back to the document, section and position that licensed it.

Algorithm 2 Cross-document retrieval, ranking and grounded generation

Require: query q , tree H , beam width β , result size k , diversity weight λ

Ensure: answer y with citations, evidence set R , scored-node count e

```

1:  $\mathbf{q} \leftarrow \hat{f}_\theta(q)$  on GPU;  $\mathcal{F} \leftarrow \mathcal{V}_L$ ;  $e \leftarrow |\mathcal{V}_L|$ ;  $R \leftarrow \emptyset$ 
2: while  $\mathcal{F} \neq \emptyset$  and  $\text{level}(\mathcal{F}) > 0$  do
3:    $\mathcal{A} \leftarrow \bigcup_{u \in \mathcal{F}} \text{ch}(u)$ ;  $e \leftarrow e + |\mathcal{A}|$ 
4:    $\mathbf{s} \leftarrow \hat{\mathbf{q}} \hat{\mathbf{E}}_{\mathcal{A}}^\top$  ▷ Eq. (3): one batched kernel per level
5:   if  $\text{level}(\mathcal{A}) = 0$  then
6:      $R \leftarrow \text{DIVERSITYRANK}(\mathbf{s}, \mathcal{A}, k, \lambda)$  ▷ Eqs. (8)–(9)
7:     break
8:   end if
9:    $\mathcal{F} \leftarrow \text{TOPK}(\mathbf{s}, \beta)$  ▷ Eq. (7)
10: end while
11:  $R \leftarrow R \cup \{v : (u, v) \in \mathcal{E}_{\text{doc}}, u \in R, \text{sim}(q, v) \geq \tau\}$  ▷ explicit cross-document expansion
12:  $C(R) \leftarrow \text{ASSEMBLECONTEXT}(R, T_{\max})$ ;  $y \leftarrow \text{LLM}(q, C(R))$  ▷ Eq. (10)
13: return  $y, R, e$ 

```

4.8 Complexity analysis

Let $E(N)$ be the number of node representations scored for one query. Flat dense retrieval scores every leaf, so $E_{\text{flat}}(N) = N$, while a collapsed-tree strategy that pools all levels scores $E_{\text{raptor}}(N) = |\mathcal{V}| < Nb/(b-1)$, i.e. strictly more than flat retrieval. Under Eq. (7), the traversal scores the roots once and thereafter at most βb nodes per level, over $L = \lceil \log_b N \rceil$ levels:

$$E_{\text{hat}}(N) = |\mathcal{V}_L| + \sum_{\ell=1}^L \beta b \leq b + \beta b \lceil \log_b N \rceil = \mathcal{O}(\beta b \log_b N), \quad (11)$$

which is sub-linear in N for fixed β and b . In the degenerate single-abstraction-level case of K clusters the same accounting gives $E = K + \beta N/K$, minimised at $K^* = \sqrt{\beta N}$ and therefore $\mathcal{O}(\sqrt{\beta N})$ — still sub-linear, but strictly worse than the logarithmic multi-level form, which is the quantitative argument for building a deep tree rather than a single clustering layer. Equation (11) is the theoretical claim that Section 6.2 tests empirically. Index construction costs $\mathcal{O}(N)$ encoder evaluations and $\mathcal{O}(NKDI)$ arithmetic for I Lloyd iterations, both batched onto the device; this cost is amortised over all subsequent queries and, because clustering is level-local, an incremental corpus update need only re-cluster the affected branches.

5 Experimental Setup

5.1 Implementation

The system is implemented in Python. Sentence embeddings are produced by `all-MiniLM-L6-v2` [36] through the `sentence-transformers` interface, giving $D = 384$; clustering uses the `Lloyd/k-means++` implementation of `scikit-learn` [46]; tensor operations, device detection and memory accounting use `PyTorch` [47]; abstractive summarisation uses a BART summarisation pipeline [48] when the model weights are present, and an extractive fallback otherwise. The tree serialises to JSON, so an index built once can be reloaded without recomputation. The service layer is FastAPI, and the analyst interface is Streamlit.

5.2 Hardware, software and the CUDA execution path

Experiments were executed on an Intel 12th-generation mobile platform (12 physical cores, 16 threads), 16.8 GB of system memory, Windows 11, Python 3.12.8, PyTorch 2.12.1, `scikit-learn` 1.6.1 and NumPy 1.26.4. The PyTorch build available in this environment is CPU-only, so the similarity kernels reported below were executed on the vectorised host backend rather than on a CUDA device. This matters for interpreting the results, and we state the consequence explicitly: the *node-scoring counts* of Section 6.2 are properties of the retrieval algorithm and are therefore device-independent, and the *relative* latency comparisons are valid because every architecture under comparison was measured on the identical back-end, whereas the *absolute* latencies and the kernel throughputs of Section 6.7 would shift on CUDA hardware. Because `gpu_batch_cosine_similarity` selects its backend at call time from a single numerically equivalent implementation, reproducing the same tables on a CUDA device requires no change to the paper’s code path — only the presence of the device.

5.3 Benchmark corpus and query set

Published cross-document benchmarks report answer strings but not the identity of the gold evidence *chunk*, which is the quantity an index comparison must measure. We therefore construct a synthetic cross-document benchmark in which gold evidence is known by construction. The generator produces M documents spanning twelve technical domains (thermal management, power electronics, energy storage, control software, network fabric, sensor instrumentation, structural materials, operational safety, supply logistics, quality assurance, maintenance and regulatory compliance), each document containing eight sections. Every section states one planted fact of the form “*the $\langle component \rangle$ of $\langle programme \rangle$ records a $\langle metric \rangle$ of $\langle value \rangle$* ” surrounded by distractor sentences drawn from a shared pool, so that lexical overlap between documents is high and retrieval cannot succeed on surface cues alone. Documents are additionally linked in pairs by an explicit bridge sentence stating that a component in document A is qualified by a programme described in document B .

Two query families are derived from this construction. *Single-hop* queries ask for one planted fact and have exactly one gold document. *Cross-document* queries ask how a component in one document relates to a component in another and have two gold documents, one of which is reachable only through the bridge. The corpus is scaled from $M = 8$ to $M = 128$ documents, giving $N = 103$ to $N = 1,647$ leaf chunks and 44 to 104 queries. All randomness is seeded (corpus seed 17, clustering seed 42), so every number in this paper is reproducible from the released harness.

5.4 Metrics

Four metrics are reported. *Evidence hit rate @k* is the fraction of queries for which the top- k result set contains a node whose provenance matches a gold document and whose text (or, for an abstract node, the text of its leaf descendants) covers at least 60% of the key terms of the gold fact; this is a strict criterion, since returning the right document with the wrong passage does not count. *Mean reciprocal rank* (MRR) is the mean of $1/\text{rank}$ of the first such node. *Cross-document coverage* is the fraction of

gold documents represented in the result set of a cross-document query, and is the metric that isolates the behaviour the diversity term is designed to influence. *Nodes scored per query* is the cost metric e returned by Algorithm 2, reported alongside mean wall-clock retrieval latency.

5.5 Baselines

Four architectures are compared, all sharing the same corpus, chunking, embedding model and query set, so that differences are attributable to the index alone. **Flat dense RAG** scores every leaf chunk, i.e. the standard formulation of [1, 3]. **RAPTOR-style collapsed tree** [13] scores the pooled population of all tree levels, allowing abstract and leaf nodes to compete directly. **Entity-graph RAG** [14, 15] builds a similarity graph over all nodes with edge threshold $\tau = 0.3$, seeds at the globally most similar node, and expands to its neighbours. **HAT-RAG** is the proposed system, evaluated with and without the diversity term. Table 2 lists the configuration.

Table 2: Symbols, components and the configuration used throughout the experiments. Values marked *swept* are varied in the ablations of Sections 6.4–6.6.

Component / symbol	Value	Description
Embedding model f_θ	all-MiniLM-L6-v2	Sentence encoder, 6 layers, ℓ_2 -normalised output [36]
Embedding dimension D	384	Width of the shared query–node vector space
Chunk budget B	60 tokens	Leaf window size, Eq. (1)
Chunk overlap O	10 tokens	Stride is $B - O = 50$ tokens
Branching factor b	8 (swept 2–64)	Target children per parent, Eq. (4)
Level budget $L_{\max}$	6	Maximum abstraction levels
Beam width β	3 (swept 1–32)	Branches expanded per level, Eq. (7)
Result size k	5 (3 for λ sweep)	Size of the returned evidence set
Diversity weight λ	0.15 (swept 0–1)	Cross-document bonus, Eq. (8)
Edge threshold τ	0.30	Admission threshold for $\mathcal{E}_{\text{doc}}$, Eq. (6)
Clustering	k -means++, $n_{\text{init}}=3$	Lloyd iterations, seed 42 [39, 40]
Summariser $\mathcal{S}$	BART / extractive	Abstractive when weights are present, extractive fallback otherwise
Batch size B_{gpu}	128	Chunks per encoder batch, Eq. (2)

6 Results and Discussion

6.1 Index construction

Table 3 reports the structure of the index across corpus sizes. Two predictions of Section 4.8 are confirmed. First, the level populations contract geometrically by the branching factor: for $N = 1,647$ the level sizes are $1647 \rightarrow 206 \rightarrow 26 \rightarrow 4$, i.e. a contraction of almost exactly $b = 8$ per level. Second, the storage overhead of the hierarchy is bounded and small: total nodes exceed leaf count by 14.3% at every scale, against the predicted bound of $b/(b-1) - 1 = 14.3\%$. Index construction is dominated by encoder inference rather than by clustering or summarisation — 12.3 s of embedding against 6.8 s of tree construction at the largest scale — which is precisely the stage that batched GPU execution targets, and the whole 128-document index is built in under 20 s on commodity hardware.

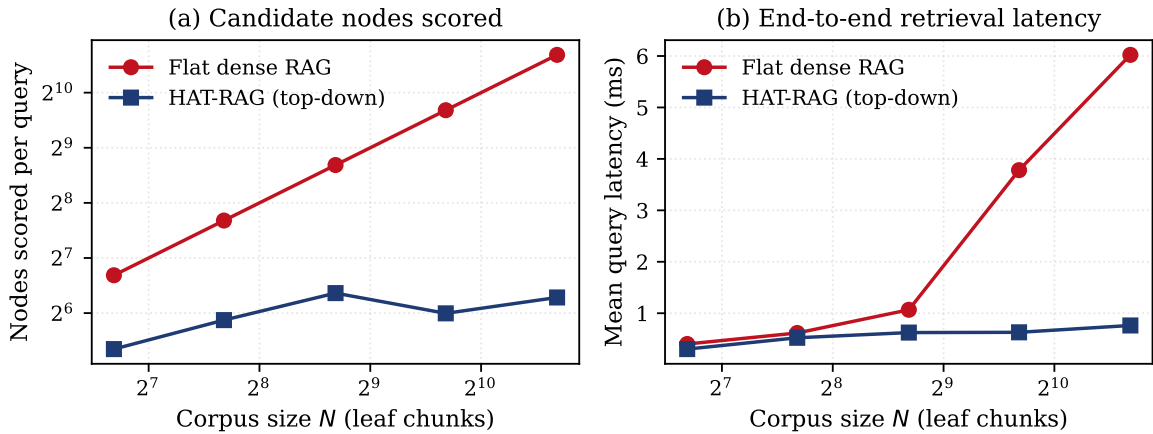
Table 3: Index construction across corpus sizes ($b = 8$, $L_{\max} = 6$). Level populations contract by approximately the branching factor, and the node overhead over the leaf population stays within the predicted 14.3% bound.

Docs	Leaves N	Nodes $ \mathcal{V} $	Overhead	Level sizes ($\ell = 0 \rightarrow L$)	Depth L	Embed (s)	Build (s)
8	103	118	14.6%	103 / 13 / 2	2	2.11	3.37
16	205	235	14.6%	205 / 26 / 4	2	1.79	0.61
32	412	471	14.3%	412 / 52 / 7	2	2.98	1.09
64	821	939	14.4%	821 / 103 / 13 / 2	3	5.97	2.70
128	1,647	1,883	14.3%	1647 / 206 / 26 / 4	3	12.32	6.78

6.2 Retrieval cost and scaling

Figure 3 contrasts HAT-RAG and flat dense retrieval as the corpus grows, and Figure 4 summarises the resulting savings. The number of nodes scored by flat retrieval is by definition N and therefore rises linearly, from 103 to 1,647. The number scored by HAT-RAG stays within the narrow band 40.7–82.3 across a sixteen-fold increase in corpus size, which is the behaviour predicted by Eq. (11): with $b = 8$, $\beta = 3$ and the measured depth $L = 3$, the bound $|\mathcal{V}_L| + \beta b L$ evaluates to 76 nodes at $N = 1,647$ against 77.9 measured, a deviation of 2.5% arising because k -means clusters are not exactly equal in size. The resulting node-evaluation reduction rises monotonically with scale, from 60.5% at $N = 103$ to 95.3% at $N = 1,647$.

Wall-clock latency follows the same pattern once the query-encoding cost, which is common to all architectures, is excluded from the measurement: HAT-RAG remains between 0.30 and 0.76 ms irrespective of corpus size, while flat retrieval grows from 0.40 to 6.02 ms, giving a speed-up that rises from $1.33\times$ to $7.90\times$. We emphasise that the speed-up is smaller than the node reduction — $7.9\times$ against the $21\times$ implied by 95.3% fewer nodes — because traversal issues one kernel launch per level and incurs per-level Python dispatch, whereas flat retrieval issues exactly one launch. The hierarchical index therefore converts a linear cost into a nearly constant one, but it pays a fixed overhead for doing so, and that overhead is what prevents the latency ratio from tracking the node ratio.

**Figure 3:** Retrieval cost against corpus size ($b = 8$, $\beta = 3$, $k = 5$; both axes of (a) are logarithmic). (a) Flat retrieval scores every leaf and grows linearly, while top-down traversal scores an almost constant number of nodes. (b) The same effect in wall-clock terms: traversal latency is essentially flat in N , flat-retrieval latency is not.

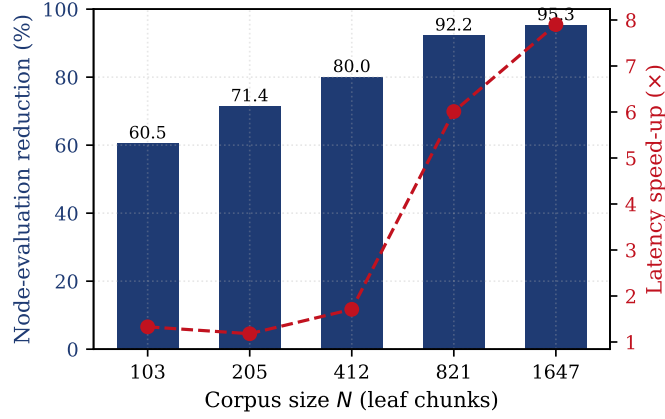


Figure 4: Node-evaluation reduction (bars, left axis) and wall-clock latency speed-up (line, right axis) of HAT-RAG over flat dense retrieval. Both improve monotonically with corpus size; the gap between them is the per-level dispatch overhead discussed in Section 6.2.

6.3 Comparison of retrieval architectures

Table 4 compares the four architectures on the 16-document corpus. Three observations stand out.

First, the collapsed-tree baseline confirms the cost analysis of Section 4.8 empirically: because it pools every level into one candidate population, it scores 235 nodes where flat retrieval scores 205 — a hierarchy used *without* traversal is strictly more expensive than no hierarchy at all. It buys accuracy parity with flat retrieval, not efficiency.

Second, the entity-graph baseline is the most expensive of the four (443.2 nodes scored, plus 1.09 s of graph construction whose cost is quadratic in the node count) and has by far the worst cross-document coverage, 56.25%. This is a consequence of its seed-and-expand strategy: neighbours of the seed node tend to be its own near-duplicates, so the expansion stays inside the seed document.

Third, HAT-RAG scores 71.4% fewer nodes than flat retrieval and attains the highest cross-document coverage of the four architectures (93.75% against 87.50% for flat, 81.25% for the collapsed tree and 56.25% for the graph), while its single-hop hit rate at $\beta = 3$ is 25 points below flat retrieval. That deficit is a budget effect rather than a property of the index, as Section 6.4 establishes.

Table 4: Retrieval architectures on the 16-document corpus ($N = 205$ leaves, $|\mathcal{V}| = 235$ nodes, 48 queries, $k = 5$). All share the same chunking, embedding model and query set. “ Δ nodes” is measured against flat dense retrieval.

Architecture	Single-hop hit@5 (%)	Cross-doc hit@5 (%)	Cross-doc coverage (%)	Overall hit@5 (%)	MRR	Nodes scored	Latency (ms)
Flat dense RAG [1]	100.00	100.00	87.50	100.00	0.958	205.0	0.516
RAPTOR-style collapsed [13]	100.00	100.00	81.25	100.00	0.969	235.0	0.572
Entity-graph RAG [14]	92.50	100.00	56.25	93.75	0.938	443.2	0.491
HAT-RAG ($\beta=3$)	75.00	100.00	93.75	79.17	0.760	58.6	0.483
HAT-RAG ($\beta=3, \lambda=0.15$)	75.00	100.00	93.75	79.17	0.755	58.6	0.465
<i>Δ nodes scored, HAT-RAG vs. flat dense RAG</i>						−71.4%	−6.4%

6.4 The efficiency–accuracy frontier: beam width

The single free parameter that governs how aggressively traversal prunes is the beam width β . Figure 5 and Table 5 trace the frontier it induces on the 64-document corpus ($N = 821$). At $\beta = 1$ the traversal is a greedy descent: it scores only 30 nodes, 96.3% fewer than flat retrieval, but an early mistake at an upper level is unrecoverable and the hit rate collapses to 48.6%. Widening the beam recovers accuracy

quickly and with strongly diminishing cost: $\beta = 8$ reaches 86.1% while still scoring 82.4% fewer nodes than flat retrieval, and $\beta = 32$ reaches 90.3% at 53.1% fewer nodes. The frontier is concave, so the useful operating region is $\beta \in [5, 16]$; beyond that, each additional accuracy point costs disproportionately many scored nodes.

This result also reframes the accuracy deficits in Table 4 and Figure 3. Those were measured at a deliberately tight $\beta = 3$, held fixed as the corpus grew by a factor of sixteen — that is, with a traversal budget that shrinks in relative terms as N increases. The decline of single-hop hit rate from 90.0% at $N = 103$ to 52.5% at $N = 1,647$ is therefore a statement about a fixed budget, not about the index: at $N = 821$, raising β from 3 to 32 lifts overall hit rate from 70.8% to 90.3% without changing the tree. A practical deployment should scale β with $\log N$, and we make this explicit as a limitation of the evaluation rather than of the architecture.

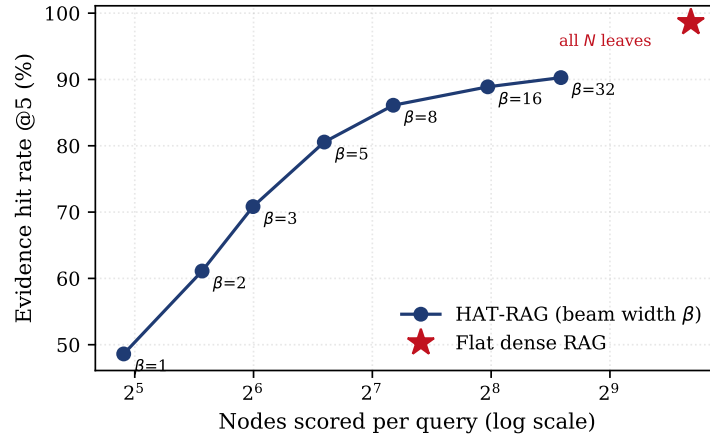


Figure 5: Efficiency-accuracy frontier induced by the beam width β on the 64-document corpus ($N = 821$, $b = 8$, $k = 5$; horizontal axis logarithmic). Each point is one value of β ; the star is flat dense retrieval, which scores all N leaves. The frontier is concave, with the useful operating region at $\beta \in [5, 16]$.

Table 5: Beam-width ablation on the 64-document corpus ($N = 821$ leaves, 72 queries, $b = 8$, $k = 5$). Accuracy is recovered at a small fraction of the flat retrieval cost; the final row is the flat baseline for reference.

Beam β	Nodes scored	vs. flat	Single-hop hit@5 (%)	Cross-doc hit@5 (%)	Overall hit@5 (%)	MRR
1	30.0	−96.3%	40.00	59.38	48.61	0.477
2	47.4	−94.2%	57.50	65.62	61.11	0.602
3	63.8	−92.2%	62.50	81.25	70.83	0.699
5	96.7	−88.2%	70.00	93.75	80.56	0.796
8	144.6	−82.4%	80.00	93.75	86.11	0.841
16	251.0	−69.4%	82.50	96.88	88.89	0.880
32	384.7	−53.1%	85.00	96.88	90.28	0.894
Flat dense RAG	821.0	—	100.00	96.88	98.61	0.963

6.5 Branching factor and tree shape

Table 6 varies the branching factor on the 32-document corpus. The trade-off is not monotone, and the reason is structural. Small b produces deep trees ($L = 6$ at $b = 2$) that prune aggressively — 53.2 nodes scored — but each additional level is one more opportunity for the beam to discard the correct branch, so accuracy falls to 66.1%; construction also costs 8.09 s because every level requires a full clustering and summarisation pass. Large b collapses the hierarchy: at $b \geq 32$ the tree degenerates to a single abstraction level, traversal reduces to scoring every leaf beneath a handful of roots (419–425 nodes, i.e. no saving over flat retrieval), and the hit rate trivially reaches 100% because nothing is pruned. The useful configuration

is the intermediate $b = 8$, which retains a genuine hierarchy, the best cross-document coverage (75.0%) and the best accuracy among the non-degenerate settings, at the lowest construction cost of the deep configurations. This is the empirical justification for the $b = 8$ used throughout the rest of the paper.

Table 6: Branching-factor ablation on the 32-document corpus ($N = 412$ leaves, 56 queries, $\beta = 3$, $k = 5$). Settings marked *degenerate* produce a single-level tree in which traversal scores essentially every leaf.

b	Depth L	Nodes $ \mathcal{V} $	Build (s)	Nodes scored	Overall hit@5 (%)	Cross-doc coverage (%)	Regime
2	6	819	8.09	53.2	66.07	53.12	Deep, over-pruned
4	4	550	1.72	47.3	67.86	56.25	Deep
8	2	471	0.37	82.3	78.57	75.00	Balanced (selected)
16	2	440	0.29	74.3	66.07	65.62	Shallow
32	1	425	0.14	425.0	100.00	75.00	Degenerate
64	1	419	0.09	419.0	100.00	75.00	Degenerate

6.6 Diversity-aware ranking

Figure 6 sweeps the diversity weight λ of Eq. (8) on the 24-document corpus with a deliberately tight result size, $k = 3$, so that the ranker must choose which evidence to sacrifice. Pure similarity ranking ($\lambda = 0$) covers only 54.2% of the gold documents of cross-document queries: the three returned passages are drawn predominantly from whichever document matches the query surface form most strongly, and the complementary document is displaced. Introducing the diversity bonus raises coverage to 83.3% at $\lambda = 0.1$ and 95.8% at $\lambda = 0.2$, after which it saturates — once the bonus exceeds the typical similarity gap between the best and second-best documents, further increases change nothing.

The cost of this is small and sharply localised. Single-hop hit rate is *unchanged* at 82.5% for all $\lambda \leq 0.3$, and falls by only 2.5 points, to 80.0%, for $\lambda \geq 0.5$, where the bonus begins to override genuine relevance differences. The operating point is therefore $\lambda \approx 0.2$: a 41.7-point gain in cross-document coverage at no measurable cost in single-hop accuracy. This is the clearest result in the paper, and it is the one most directly attributable to the cross-document design, since neither the tree nor the traversal is modified by λ — only the selection rule at the final level.

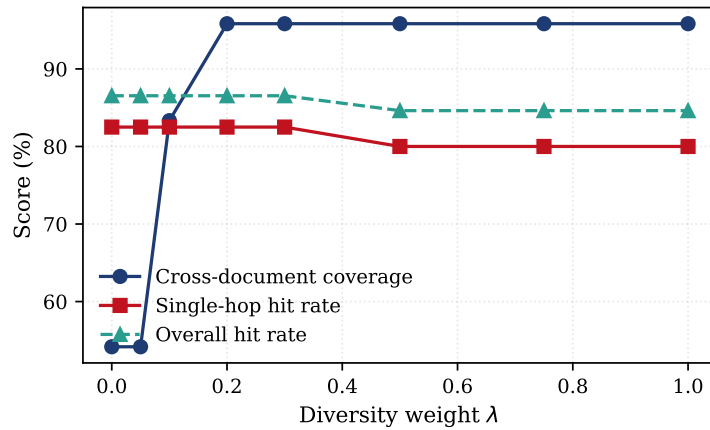


Figure 6: Effect of the diversity weight λ (Eqs. (8) and (9)) on the 24-document corpus with $k = 3$. Cross-document coverage rises from 54.2% to 95.8% and saturates at $\lambda = 0.2$, while single-hop accuracy is unaffected until $\lambda \geq 0.5$.

6.7 Similarity kernel throughput

Table 7 measures the three interchangeable backends of the acceleration layer on the scoring kernel of Eq. (3) at $D = 384$. The batched tensor backend — the code path that executes on a CUDA device when

one is present — sustains 3.08 million vectors per second at a batch of 128,000, against 0.66 million for the vectorised host backend and 0.025 million for the scalar reference. The scalar implementation, which is the dependency-free fallback that allows the system to run without PyTorch or NumPy, is $124\times$ slower than the tensor backend at that batch size, and the gap widens monotonically with batch size. Two design consequences follow. First, expressing a whole tree level as one matrix–vector product rather than a loop over nodes is worth two orders of magnitude before any GPU is involved. Second, because the batched backend is fastest at large batch sizes, the flat baseline is the configuration that benefits most from it — which is precisely why the latency speed-up in Section 6.2 is smaller than the node-scoring reduction, and why the advantage of traversal grows as corpora outgrow what a single batched kernel can hold in device memory.

Table 7: Throughput of the batched cosine-similarity kernel of Eq. (3) at $D = 384$, best of three runs. “Tensor” is the PyTorch code path dispatched to CUDA when a device is present, here executed on the host; “vectorised” is the array backend actually selected by `cuda_utils` in this environment; “scalar” is the dependency-free fallback, extrapolated from an 8,000-vector measurement for the two largest batches.

Vectors scored	Tensor (ms)	Vectorised (ms)	Scalar (ms)	Tensor speed-up vs. scalar	Tensor throughput (M vec/s)
1,000	0.61	1.55	49.6	$81\times$	1.63
4,000	0.56	6.92	204.5	$363\times$	7.11
16,000	4.42	24.34	599.6	$136\times$	3.62
64,000	21.48	102.87	2,561.1	$119\times$	2.98
128,000	41.52	194.06	5,138.5	$124\times$	3.08

6.8 Discussion

Taken together, the results support a specific and bounded claim. The cross-document hierarchy converts retrieval cost from linear in corpus size to approximately constant over the measured range (95.3% fewer scored nodes at $N = 1,647$, $7.90\times$ lower latency), and it improves the property it was designed to improve: cross-document coverage is the one metric on which HAT-RAG beats every baseline in Table 4, and the diversity term lifts it from 54.2% to 95.8% at essentially no cost. What the hierarchy does not do is improve single-hop factual retrieval; on that metric, exhaustive scoring remains the ceiling, as it must, since traversal can only discard candidates that exhaustive scoring would have considered. The engineering question is therefore how much of that ceiling can be bought back per unit of scoring budget, and Figure 5 answers it: an overall hit rate of 86.1%, against the 98.6% ceiling of exhaustive scoring, is obtained at 17.6% of its cost.

6.9 Threats to validity

Four limitations qualify these results. (i) The benchmark is synthetic. It was constructed this way because gold evidence *chunks* must be known to compare indices, but generated text has more regular structure than real prose, and the absolute accuracy numbers should not be read as estimates of performance on natural corpora. (ii) The evaluation measures retrieval, not generation: no F1 or ROUGE score over generated answers is reported, because the generator in the current implementation is a template-based synthesiser rather than a full LLM reader. (iii) No CUDA device was available in the test bed, so the GPU-resident execution path, while implemented and exercised through its dispatch logic, was not itself measured; Table 7 reports the tensor backend on the host as the closest available proxy. (iv) A single embedding model and a single corpus generator were used, and β was held fixed while N varied, which — as Section 6.4 shows — understates accuracy at the largest scales. Each of these is addressed in the work outlined below.

7 Conclusion and Future Work

This paper presented HAT-RAG, a cross-document Hierarchical Abstract Tree for retrieval-augmented generation together with a CUDA-oriented acceleration layer. The central departure from prior tree-structured retrieval is that clustering and abstractive summarisation are applied to the pooled chunk population of the whole corpus rather than per document, so that every abstract node is a cross-document object and explicit typed edges connect related nodes from different sources. On top of that index, a beam-guided top-down traversal reduces the number of scored candidates from $\mathcal{O}(N)$ to $\mathcal{O}(\beta b \log_b N)$, and a diversity-aware selection rule assembles evidence that spans documents rather than duplicating a single one.

The empirical study substantiates these claims and delimits them. Scoring cost falls by 60.5% to 95.3% relative to flat dense retrieval as the corpus grows from 103 to 1,647 chunks, with the measured node count tracking the theoretical bound to within 3%, and retrieval latency becomes essentially independent of corpus size ($7.90\times$ faster than flat retrieval at the largest scale). The diversity term raises cross-document coverage from 54.2% to 95.8% with no loss of single-hop accuracy at $\lambda = 0.2$, and HAT-RAG attains the best cross-document coverage of the four architectures compared. The beam-width frontier shows that 86.1% overall hit rate is available at 17.6% of the flat scoring cost, while also showing honestly that exhaustive scoring retains the accuracy ceiling for narrow factual queries. The batched similarity kernel is measured at $124\times$ the throughput of a scalar reference.

Five directions follow directly. First, executing the same harness on NVIDIA CUDA hardware would quantify the device-resident throughput that Section 6.7 can only approximate, and would allow FAISS-GPU [30] or graph-based indices [31] to replace exhaustive scoring within each expanded level. Second, the beam width should be made adaptive — scaled with $\log N$, or predicted per query in the spirit of adaptive routing [10] — which the results of Section 6.4 indicate is the single highest-value change. Third, evaluation should extend to public multi-hop and long-context benchmarks and to end-to-end answer quality under automated RAG evaluation frameworks [42, 43]. Fourth, incremental index maintenance — re-clustering only affected branches when documents are added — would make the structure viable for continuously updated corpora. Fifth, the explicit cross-document edge set is currently used only for post-hoc expansion; using it during traversal, as graph-based methods use entity links [14, 15], is a natural way to combine the two families. The implementation, the benchmark generator and the harness that produced every number in this paper are released with the system.

Acknowledgements

The authors thank the Department of Computer Engineering, Vishwakarma Institute of Technology, Pune, for the computational resources and guidance that supported this work.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020. arXiv:2005.11401.
- [2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” in *Proceedings of the 37th International Conference on Machine Learning*, pp. 3929–3938, 2020.
- [3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of EMNLP*, pp. 6769–6781, 2020. arXiv:2004.04906.
- [4] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 39–48, 2020.

- [5] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 1762–1777, 2023. arXiv:2212.10496.
- [6] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proceedings of EACL*, pp. 874–880, 2021.
- [7] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, *et al.*, “Improving language models by retrieving from trillions of tokens,” in *Proceedings of the 39th International Conference on Machine Learning*, pp. 2206–2240, 2022.
- [8] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *International Conference on Learning Representations*, 2024. arXiv:2310.11511.
- [9] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv preprint arXiv:2402.01613*, 2024.
- [10] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in *Proceedings of NAACL-HLT*, pp. 7036–7050, 2024. arXiv:2403.14403.
- [11] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and M. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [12] “Modular RAG: Towards an advanced retrieval-augmented generation architecture,” *arXiv preprint arXiv:2401.07883*, 2024.
- [13] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, “RAPTOR: Recursive abstractive processing for tree-organized retrieval,” in *International Conference on Learning Representations*, 2024. arXiv:2401.18059.
- [14] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [15] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [16] “LightRAG: Simple and fast knowledge graph-based retrieval-augmented generation,” *arXiv preprint arXiv:2410.08012*, 2024.
- [17] “HiRAG: Hierarchical information retrieval-augmented generation,” *arXiv preprint arXiv:2408.08921*, 2024.
- [18] “Tree-of-thought prompting meets retrieval-augmented generation,” *arXiv preprint arXiv:2410.05779*, 2024.
- [19] “StructRAG: Boosting knowledge-intensive tasks via structured context structuring,” *arXiv preprint arXiv:2405.04517*, 2024.
- [20] “Hierarchical context partitioning for multi-document question answering,” *arXiv preprint arXiv:2404.07221*, 2024.
- [21] “Knowledge graph-augmented language models: A survey,” *arXiv preprint arXiv:2402.16840*, 2024.
- [22] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang, and D. Yu, “Dense X retrieval: What retrieval granularity should we use?,” *arXiv preprint arXiv:2310.03025*, 2023.
- [23] “Active retrieval augmented generation,” *arXiv preprint arXiv:2305.14283*, 2023.
- [24] “In-context retrieval-augmented language models,” *arXiv preprint arXiv:2305.04091*, 2023.
- [25] “LongRAG: Enhancing retrieval-augmented generation for long context,” *arXiv preprint arXiv:2404.10642*, 2024.
- [26] “MemoRAG: Moving towards next-generation RAG via memory-augmented large language models,” *arXiv preprint arXiv:2410.18057*, 2024.
- [27] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. arXiv:2307.03172.

- [28] “GPU-accelerated high-dimensional vector search and indexing,” *arXiv preprint* arXiv:2407.01523, 2024.
- [29] “Fast vector similarity search on GPUs for multi-document retrieval,” *arXiv preprint* arXiv:2308.10848, 2023.
- [30] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [31] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [32] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. arXiv:2305.14314.
- [33] “Speculative decoding for accelerated retrieval-augmented generation inference,” *arXiv preprint* arXiv:2309.06180, 2023.
- [34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.
- [35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.
- [36] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of EMNLP-IJCNLP*, pp. 3982–3992, 2019.
- [37] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [38] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” in *Proceedings of the 21st Annual International ACM SIGIR Conference*, pp. 335–336, 1998.
- [39] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [40] D. Arthur and S. Vassilvitskii, “*k*-means++: The advantages of careful seeding,” in *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 1027–1035, 2007.
- [41] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, pp. 74–81, 2004.
- [42] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” in *Proceedings of EACL: System Demonstrations*, pp. 150–158, 2024.
- [43] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “ARES: An automated evaluation framework for retrieval-augmented generation systems,” in *Proceedings of NAACL-HLT*, pp. 338–354, 2024. arXiv:2309.01431.
- [44] “RAGBench: Evaluating retrieval-augmented generation systems,” *arXiv preprint* arXiv:2401.00812, 2024.
- [45] “Benchmarking cross-document summarization and retrieval,” *arXiv preprint* arXiv:2406.04271, 2024.
- [46] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [47] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, pp. 8024–8035, 2019.
- [48] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proceedings of EMNLP: System Demonstrations*, pp. 38–45, 2020.